PROJECT

{ INVENTORY MANAGEMENT }

Members :

1. Abdul Wahid [024]
2. Saeed Shah [032]
3. Adan Kaleem [059]

Detailed Report

1. Overview

   The **Inventory Management System** is a console-based Java application designed to manage inventory operations for a company.
   It allows the **Manager** to overseer multiple **Warehouses** , each containing **Products** and interacts with **Suppliers.**

   The system demonstrates key Object-Oriented Programming (OOP) principles such as:

   i. **Inheritance**
   ii. **Compisition and Aggregation**

iii. **Encapsulation**
Constructor Chaining
iv. **Use of final keyword**
v. **Access modifiers**
(protected, private, public)

2. Objective

The main objectives of this project are:

- To implement **OOP concepts** effectively in a single Java package (project)
- To design a system that manages inventory items, warehouses, and suppliers
- To demonstrate **Has–A relationships** (Composition & Aggregation) between entities
- To show **Is–A relationships** through inheritance (Manager → Person)
- To use final, protected, and constructor chaining appropriately

3. Package and Class Structure

```
package project
    |
    |------ AppConstants.java  (Final constants class)
    |------ Person.java (Superclass)
```

|------ Manager.java (**Subclass** of Person)

|------ Product.java (**Entity class)**

|------ Supplier.java (**Aggregation class)**

|------ Warehouse.java (**Composition class**)

|------ InventorySystem.java (**Aggregator and controller class**)

|------ Inventory_Management_System.java (**Main class**)

4. Class Description

(1) AppConstants

- **Type:** final class
- **Purpose:** Defines immutable constants shared across the application.
- **Key Feature:**
  - final class prevents inheritance.
  - Contains a private constructor to prevent instantiation

(2) Person (Superclass)

- **Purpose:** Represents a general person with basic attributes.
- **Access:** protected for inheritance flexibility.
- **Key Features:**
  - Demonstrates **encapsulation** via getters/setters.
  - Used as a **base class** for the Manager.

(3) Manager (Subclass of Person)

- **Purpose:** Represents the system's administrator responsible for inventory operations.
- **Relationships:**
  - **Is–A** → Manager *is a* Person.
- **OOP Features:**
  - **Constructor Chaining** → calls superclass constructor using super(name, id).
  - **final Method:** auditInventory() can't be overridden.
  - **Method Overriding:** Extends functionality with displayInfo().

(4) Product

- **Purpose:** Represents an item stored in a warehouse.
- **Key Features:**
  - final String sku → ensures product SKU cannot change once created.
  - Encapsulated fields with getters/setters.
  - display() method for formatted product info.
- **Composition:** Strongly tied to Warehouse (dies when warehouse is deleted).

(5) Supplier

- **Purpose:** Represents an external supplier linked with the inventory.
- **Relationship:**

- **Aggregation** with InventorySystem (suppliers exist independently).

(6) Warehouse

- **Purpose:** Represents storage facilities for products.
- **Composition:**
  - **Warehouse has–a Product** (Products exist *within* Warehouse).
- **Key Methods:**
  - addProduct(Product p)
  - removeProduct(String sku)
  - findProduct(String sku)
  - listAllProducts()
  - searchByName(String namePart)

(7) InventorySystem

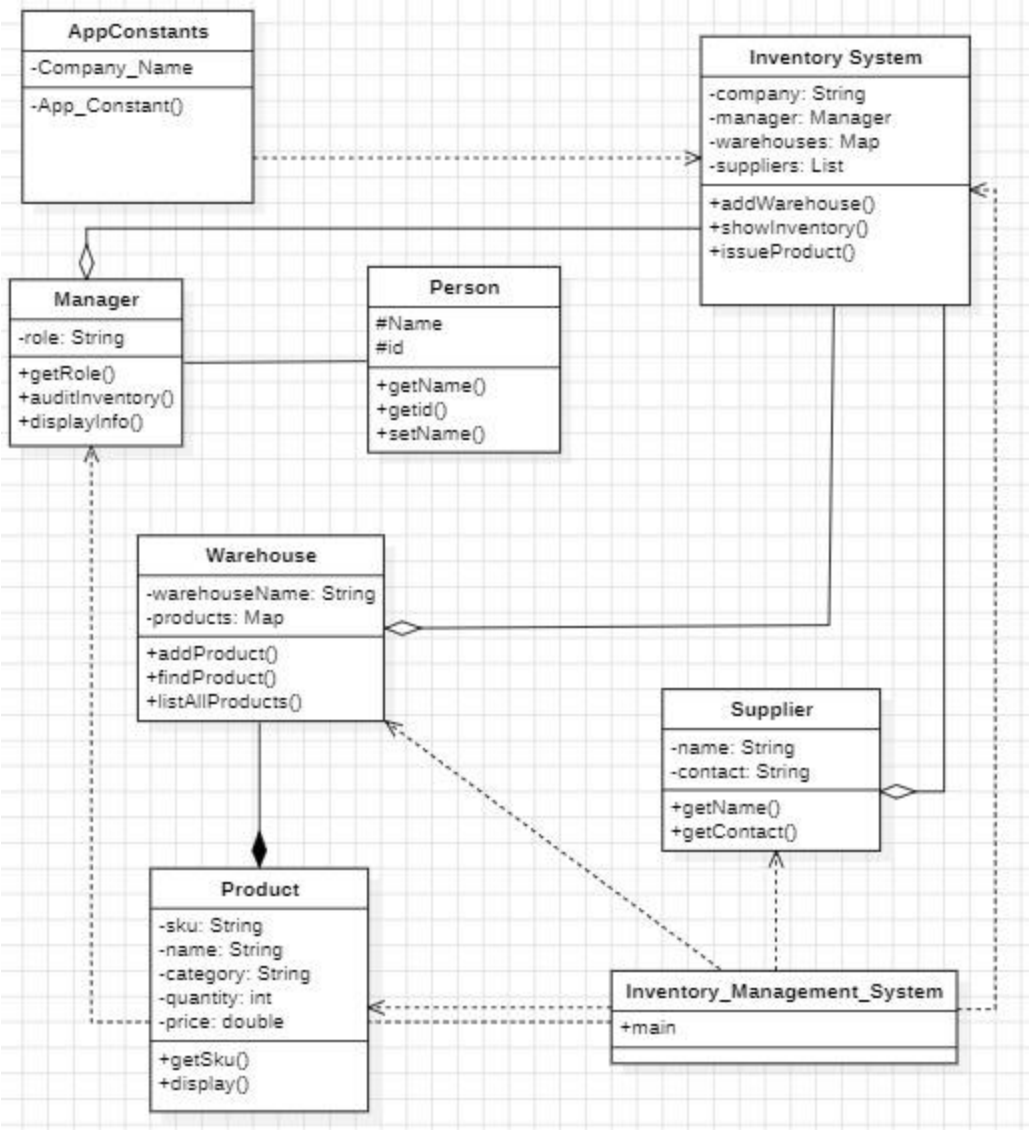
- **Purpose:** Acts as the **central controller**, managing warehouses, suppliers, and inventory operations.
- **Aggregation:**
  - Has multiple **Warehouses**.
  - Has multiple **Suppliers**.
  - Has one **Manager**.
- **Key Functionalities:**
  - Add/remove warehouses.

- Issue and receive products.
- Display full inventory.
- Search across warehouses.

(8) Inventory_Management_System (Main Class)

- **Purpose:** Entry point of the application.
- **Features:**
  - Provides a **menu-driven interface** using Scanner.
  - Supports:
    I. Viewing inventory
    II. Issuing products
    III. Receiving stock
    IV. Searching for products
    V. Exiting the system

5. UML Diagram



6. Key Advantages

- Modular and **extensible** system.
- Demonstrates all core **OOP principles** clearly.
- Easy to integrate with **databases or GUIs** in future.
- **Menu-driven** interface for easy interaction.

7. Conclusion

This project effectively demonstrates the **object-oriented design principles** within a realistic scenario.

By combining **composition, aggregation, inheritance, and encapsulation**, it models a maintainable and extensible **Inventory Management System** that can be scaled for enterprise use.